ADR-002: 采用Spring Boot + Vue.js技术栈

状态

已采纳

背景

项目需求

校园二手交易平台需要实现以下核心功能:

- 用户认证与授权
- 商品发布、浏览、搜索
- 订单管理与交易
- 评价系统
- 消息通知
- 收藏功能

技术选型约束

1. **开发周期**: 10周内完成可运行系统
2. **团队背景**: 团队成员对Java和JavaScript有基础
3. **项目规模**: 中小型Web应用
4. **部署环境**: 需要支持快速部署和演示

需要解决的问题

如何选择合适的技术栈,以满足:

1. 快速开发和迭代
2. 良好的开发体验
3. 充足的社区支持和文档
4. 团队学习成本可控
5. 适合前后端分离架构

决策

本项目采用以下技术栈:

后端技术栈

技术	版本	用途
Spring Boot	3.3.5	后端框架,提供快速开发和自动配置
Java	17	编程语言,LTS版本
MyBatis Plus	3.5.7	ORM框架,简化数据库操作
MySQL	8.0.32	关系型数据库
Lombok	1.18.26	简化Java代码,自动生成getter/setter
Swagger	1.5.1	API文档生成

前端技术栈

技术	版本	用途
Vue.js	3.5.32	前端框架,响应式UI开发
Vite	8.0.4	构建工具,提供快速开发服务器

技术	版本	用途
JavaScript	ES6+	编程语言

决策理由

后端选择Spring Boot的原因

1. **快速开发**: Spring Boot提供自动配置和starter依赖,大幅减少配置工作
2. **生态成熟**: Spring生态系统完善,有丰富的解决方案
3. **团队熟悉**: 团队成员有Java基础,学习曲线平缓
4. **文档完善**: 官方文档和社区资源丰富,遇到问题容易找到解决方案
5. **适合Web开发**: Spring MVC天然支持RESTful API开发
6. **企业级特性**: 内置安全、事务、监控等企业级特性

选择MyBatis Plus的原因

1. **简化开发**: 相比原生MyBatis,MyBatis Plus提供CRUD接口,减少重复代码
2. **灵活性强**: 支持自定义SQL,适合复杂查询
3. **代码生成**: 提供代码生成器,快速生成Entity、Mapper、Service
4. **性能优化**: 内置分页插件、性能分析插件

前端选择Vue.js的原因

1. **学习曲线平缓**: Vue.js语法简洁,容易上手
2. **组件化开发**: 支持组件化,便于代码复用和维护
3. **响应式系统**: 自动追踪依赖,简化状态管理
4. **生态完善**: Vue Router、Vuex/Pinia等配套工具完善
5. **性能优秀**: 虚拟DOM和编译优化,渲染性能好
6. **文档友好**: 中文文档完善,适合国内开发者

选择Vite的原因

1. **开发体验**: 极速的开发服务器启动和热更新
2. **构建性能**: 基于ES模块,构建速度快
3. **配置简单**: 开箱即用,配置灵活
4. **现代化**: 支持最新的JavaScript特性

后果

积极影响

1. **开发效率高**: Spring Boot和Vue.js都强调快速开发,适合10周项目周期
2. **学习成本低**: 团队已有Java和JavaScript基础,学习曲线平缓
3. **社区支持好**: 两个框架都有活跃的社区和丰富的文档
4. **代码质量**: Lombok和MyBatis Plus减少样板代码,提高代码可读性
5. **开发体验**: Vite提供快速的开发服务器,提升开发体验
6. **API文档**: Swagger自动生成API文档,便于前后端协作

消极影响

1. **框架学习**: 团队需要学习Spring Boot和Vue.js的新特性
2. **版本兼容**: Spring Boot 3需要Java 17+,需要注意版本兼容性
3. **配置复杂度**: Spring Boot虽然简化配置,但某些场景仍需深入理解
4. **前端状态管理**: Vue 3的组合式API需要团队适应

风险

1. **版本更新风险**: Spring Boot 3和Vue 3都是较新版本,可能遇到未知问题
    - **缓解措施**: 使用稳定的LTS版本,及时关注官方更新和社区反馈

2. **性能风险**: ORM框架可能产生N+1查询问题
    - **缓解措施**: 使用MyBatis Plus的性能分析插件,进行SQL优化
3. **安全风险**: Web应用面临XSS、CSRF等安全威胁
    - **缓解措施**: 使用Spring Security进行安全控制,前端进行输入验证

替代方案

后端替代方案

方案A: Node.js + Express

拒绝原因:

1. 团队对Java更熟悉
2. Spring Boot生态更成熟,企业级特性更完善
3. 类型安全:Java的类型系统有助于减少运行时错误

方案B: Django (Python)

拒绝原因:

1. 团队Python经验较少
2. 需要学习新的语言和框架,增加学习成本
3. 与团队技术栈不一致

方案C: Spring Cloud微服务

拒绝原因:

1. 与ADR-001决策冲突,本项目不采用微服务架构
2. 增加系统复杂度,不适合10周项目周期
3. 团队缺乏微服务经验

前端替代方案

方案A: React

拒绝原因:

1. 学习曲线比Vue陡峭
2. 需要额外学习JSX语法
3. 团队对Vue更感兴趣

方案B: Angular

拒绝原因:

1. 框架过于庞大,学习成本高
2. TypeScript强制要求,增加学习负担
3. 不适合中小型项目快速开发

方案C: 传统服务端渲染(Thymeleaf/JSP)

拒绝原因:

1. 不符合前后端分离的架构决策
2. 用户体验不如SPA应用

3. 前后端耦合度高,不利于团队协作

技术栈架构图

```
┌──────────────────────────────────────────────────────────────┐
│                          前端层                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  Vue 3 + Vite                                        │    │
│  │  - 组件化UI                                          │    │
│  │  - 状态管理                                          │    │
│  │  - 路由管理                                          │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
                            │ HTTP/REST API
                            ↓
┌──────────────────────────────────────────────────────────────┐
│                          后端层                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  Spring Boot 3.3.5                                   │    │
│  │  ┌──────────────┐┌──────────────┐┌──────────────┐  │    │
│  │  │ Controller   ││   Service    ││  Repository  │  │    │
│  │  │   层         ││    层        ││    层        │  │    │
│  │  └──────────────┘└──────────────┘└──────────────┘  │    │
│  │                                                      │    │
│  │  ┌──────────────────────────────────────────┐       │    │
│  │  │  MyBatis Plus 3.5.7                      │       │    │
│  │  └──────────────────────────────────────────┘       │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
                            │ JDBC
                            ↓
┌──────────────────────────────────────────────────────────────┐
│                        数据库层                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  MySQL 8.0.32                                        │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

实施计划

阶段1: 环境搭建(第1周)

1. 安装JDK 17、Maven、MySQL
2. 创建Spring Boot项目,配置依赖
3. 创建Vue项目,配置Vite
4. 配置数据库连接

阶段2: 后端开发(第2-6周)

1. 设计数据库表结构
2. 使用MyBatis Plus生成基础代码
3. 实现Controller、Service、Mapper层
4. 配置Swagger API文档
5. 编写单元测试

阶段3: 前端开发(第2-6周)

1. 设计页面原型
2. 创建Vue组件
3. 实现页面交互逻辑
4. 对接后端API
5. 前端测试

阶段4: 集成测试(第7-8周)

1. 前后端集成测试
2. 性能测试和优化
3. 安全测试

阶段5: 部署与交付(第9-10周)

1. 打包部署
2. 编写部署文档
3. 准备演示材料

技术规范

后端规范

1. **代码规范**: 遵循阿里巴巴Java开发手册
2. **命名规范**: 类名使用大驼峰,方法名和变量名使用小驼峰
3. **注释规范**: 使用Javadoc注释公共方法
4. **异常处理**: 使用全局异常处理器,统一返回错误信息
5. **日志规范**: 使用SLF4J,合理设置日志级别

前端规范

1. **代码规范**: 遵循Vue官方风格指南
2. **命名规范**: 组件名使用大驼峰,其他使用小驼峰
3. **目录结构**: 按功能模块组织代码
4. **状态管理**: 简单状态使用reactive,复杂状态考虑Pinia
5. **样式规范**: 使用Scoped CSS,避免样式污染

参考

- Spring Boot官方文档: https://spring.io/projects/spring-boot
- Vue.js官方文档: https://vuejs.org/
- MyBatis Plus官方文档: https://baomidou.com/
- Vite官方文档: https://vitejs.dev/

决策历史

- 2026-05-01: 团队讨论并确定技术栈选型
- 2026-05-01: 正式采用Spring Boot + Vue.js技术栈